Lycée Duhamel du Monceau. Niveau : Première.		
	Numérique et Sciences Informatiques	
Diversité et unité des langages de programmation		

I) <u>INTRODUCTION</u> :

Selon Wikipédia, un langage de programmation est une notation conventionnelle destinée à formuler des algorithmes et produire des programmes informatiques qui les appliquent. D'une manière similaire à une langue naturelle, un langage de programmation est composé d'un alphabet, d'un vocabulaire, de règles de grammaire et de significations. Les langages de programmation permettent de décrire d'une part les structures des données qui seront manipulées par l'appareil informatique, et d'autre part d'indiquer comment sont effectuées les manipulations, selon quels algorithmes. Ils servent de moyens de communication par lesquels le programmeur communique avec l'ordinateur.

Un langage de programmation est mis en œuvre par un traducteur automatique : compilateur ou interprète. Un compilateur est un programme informatique qui transforme dans un premier temps un code source écrit dans un langage de programmation donné en un code cible qui pourra être directement exécuté par un ordinateur, à savoir un programme en langage machine ou en code intermédiaire, tandis que l'interprète réalise cette traduction « à la volée ». Les langages de programmation offrent différentes possibilités d'abstraction, et une notation proche de l'algèbre, permettant de décrire de manière concise et facile à saisir les opérations de manipulation de données et l'évolution du déroulement du programme en fonction des situations. La possibilité d'écriture abstraite libère l'esprit du programmeur d'un travail superflu, notamment de prise en compte des spécificités du matériel informatique, et lui permet ainsi de se concentrer sur des problèmes plus avancés.

Chaque langage de programmation supporte une ou plusieurs approches de la programmation – *paradigmes*. Les notions induisant le paradigme font partie du langage de programmation et permettent au programmeur d'exprimer dans le langage une solution qui a été imaginée selon ce paradigme.
Les premiers langages de programmation ont été créés dans les années 1950 en même temps que l'avènement des ordinateurs. Cependant, de nombreux concepts de programmation ont été initiés par un langage ou parfois plusieurs langages, avant d'être améliorés puis étendus dans les langages suivants. La plupart du temps la conception d'un langage de programmation a été fortement influencée par l'expérience acquise avec les langages précédents.

II) <u>OBJECTIF DE CETTE SEQUENCE</u> :

Dans cette séquence, nous allons étudier les particularités du **langage C** et faire un parallèle avec un langage que nous avons déjà étudié : le **langage Python**.
Le but sera de mettre en évidence les points communs de ces 2 langages mais aussi et surtout leurs principales différences.

Le **langage Python** est un langage de programmation de haut niveau utilisé pour les sites Internet et les applications mobiles. Il est considéré comme un langage plutôt facile à maîtriser pour les débutants grâce à sa syntaxe compacte et parce qu'il est très lisible, ce qui permet aux développeurs d'utiliser moins de lignes de code pour exprimer une fonction qui en aurait nécessité plus dans d'autres langages. Il fait fonctionner les applications Web d'**Instagram**, **Pinterest** et est utilisé par **Google**, **Yahoo,** et même la **NASA**.

Le **langage C** est le langage de programmation le plus vieux et un des plus utilisés au monde, créé dans les années 70. Il pose les bases de nombreux autres langages, comme le **C++**, le **C#**, le **Java**… il est donc conseillé de s'y intéresser de près. Le **C** est principalement utilisé pour intégrer des systèmes d'opérations et des applications incrustées dans des pages Web.

III) <u>LE LANGAGE C VS LE LANGAGE PYTHON</u> :

Langage C	Langage Python
C est un langage compilé, il est donc très rapide.	Python est un langage interprété, son exécution est donc lente.

En C le programmeur doit gérer lui-même la mémoire ce qui est contraignant.	Python gère automatiquement la mémoire.
En C il est indispensable de déclarer chaque type de variable.	En Python, inutile de déclarer le type de variable, ceci est fait automatiquement.
Le langage C est un langage structuré.	Python est un langage multi-paradigmes.

IV) UN PEU DE COURS :

Nous ne nous attarderons que sur les principales différences existant entre le langage C et le Python. Pour avoir d'autres informations sur C, se reporter aux différents documents présents sur le site, ou sur internet.

1°) Types de variables :

Nous l'avons vu, **en C il va falloir déclarer pour chaque variable son type**, **ceci est indispensable**.
En C, les principaux types de variables sont les suivants :

Type de donnée	Signification	Taille (en octets)	Plage de valeurs acceptée
char	Caractère	1	-128 à 127
unsigned char	Caractère non signé	1	0 à 255
short int	Entier court	2	-32 768 à 32 767
unsigned short int	Entier court non signé	2	0 à 65 535
int	Entier	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned int	Entier non signé	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	0 à 65 535 0 à 4 294 967 295
long int	Entier long	4	-2 147 483 648 à 2 147 483 647
unsigned long int	Entier long non signé	4	0 à 4 294 967 295
float	Flottant (réel)	4	$-3.4*10^{38}$ à $3.4*10^{38}$
double	Flottant double	8	$-1.7*10^{308}$ à $1.7*10^{308}$
long double	Flottant double long	10	$-3.4*10^{4932}$ à $3.4*10^{4932}$

Il ne faudra surtout pas se tromper lors de la déclaration du type de la variable car sinon il pourrait y avoir des erreurs par débordement.

Pour chacun des exemples ci-dessous, dites si le type convient :

```
int a=6 ;

float a1=3.11 ;

char b=-23 ;

unsigned char a2=250 ;

float a3=5.0 ;

int a4=63256 ;

double a5=-523254 ;
```

On notera qu'il faut toujours terminer une instruction en C avec <u>un point-virgule</u>.

2°) <u>Les fonctions</u> :

Ce qui suit est à tester dans Code::Blocks.

Ecrire une fonction :

Syntaxe :

```
type identificateur(paramètres)
{
   Corps de la fonction
}
```

A chaque appel de la fonction on exécute le corps de la fonction.
L'identificateur est le nom de la fonction.
La fonction peut avoir des paramètres.
La fonction peut renvoyer une valeur de types différents.

Appel à une fonction :

Lors de l'appel de la fonction, le programme exécute la totalité des instructions du corps de la fonction, puis reprend le programme juste après l'appel de la fonction.

Un exemple de fonction :

```
#include <stdio.h>
#include <stdlib.h>

void b()
{
   printf("Bonjour\n");
}
int main()
{
   printf("COUCOU1\n");
   b();
   printf("COUCOU2\n");
   b();
   b();
   printf("COUCOU3\n");
   b();
   return 0;
}
```

Explications :

• stdio.h et stdlib.h sont les fichiers 'headers' dans lesquelles se trouvent les prototypes des fonctions standard, comme printf, scanf, puts, fputc, sscanf, etc. Ces fonctions ne sont pas dans le langage C par défaut.
Ces fonctions ont été mises dans une library (.a ou .lib), en incluant le header (.h), on dit au compilateur quelles fonctions on souhaite utiliser.
#include <stdio.h> == Je veux utiliser les fonctions du header stdio.
• Dans ce programme, on a créé une fonction b qui se contente d'afficher "Bonjour" à l'écran avec l'instruction printf.
La fonction b est précédée du type void : cela signifie que la fonction ne renvoie aucune valeur au programme appelant.
• \n permet un retour à la ligne.
• Le programme principal (la fonction main()) affiche "COUCOU1" à l'écran, ensuite appelle la fonction b, affiche le message "COUCOU2" à l'écran, appelle ensuite 2 fois la fonction b, affiche le message "COUCOU3" et appelle une dernière fois la fonction b.

Exécution :

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
COUCOU1
Bonjour
COUCOU2
```

```
Bonjour
Bonjour
COUCOU3
Bonjour
```

Les paramètres :
On peut paramétrer une fonction : les paramètres permettent de rendre la fonction plus générale donc plus facilement réutilisable. La réutilisation de code est une des notions fondamentales du langage C. La notion de fonction est une des premières méthodes nous permettant d'aborder le sujet.

Une fonction avec des paramètres :

```
#include <stdio.h>
#include <stdlib.h>

void b(int i)
{
   int j;
   for (j=0;j<i;j++) printf("Bonjour\n");
}
int main()
{
   printf("COUCOU1\n");
   b(2);
   printf("COUCOU2\n");
   b(3);
   return 0;
}
```

Explications :
• Cette fois-ci la fonction b comporte un paramètre entier i. Cette fonction affiche i fois "Bonjour" à l'écran.
• Dans le programme principal, la fonction main() affiche "COUCOU1", appelle ensuite la fonction b avec comme paramètre 2.
• Lors de l'appel b(2) de la fonction, on recopie la valeur 2 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 2 fois "Bonjour".
• Lors de l'appel b(3) de la fonction, on recopie la valeur 3 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 3 fois "Bonjour".

Exécution :
Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
COUCOU1
Bonjour
Bonjour
COUCOU2
Bonjour
Bonjour
Bonjour
```

Environnement d'une fonction :
• Une fonction peut avoir ses propres variables locales.
• On appelle environnement d'une fonction l'ensemble des variables accessibles dans le corps de la fonction.
• Il est possible de définir des variables globales (voir la définition des constantes) : il est fortement déconseillé d'utiliser des variables globales autres que des constantes.
• L'environnement d'une fonction comprend :
    • les variables locales à la fonction.
    • les paramètres de la fonction.
    • les variables globales.

Une fonction qui renvoie une valeur :

Une fonction peut renvoyer une valeur d'un certain type à l'environnement appelant. **void** signifie que la fonction ne renvoie rien. Si on veut que la fonction renvoie une certaine valeur, il faudra écrire un autre type à la place de void.
Le mot clé **return** permet :
• d'arrêter l'exécution de la fonction (il est en général placé à la fin de la fonction).
• de renvoyer une valeur à l'environnement appelant.
Si une fonction b renvoie un entier, l'appel s'écrira : a=b(…); a permettra de récupérer la valeur renvoyée par le return.

Exemple de fonction qui renvoie une valeur :

```
#include <stdio.h>
#include <stdlib.h>

int b(int i, int j)
{
   int k;
   k = i*i + j*j;
   return k;
}
int main()
{
   int a;
   a = b(3,4);
   printf("Le resultat vaut: %i\n",a);
   return 0;
}
```

Explications :
Lors de l'appel **a=b(3,4);**
• 3 et 4 sont respectivement recopiés dans les variables i et j.
• On exécute le corps de la fonction (qui contient une variable locale k).
• **return k** permet d'arrêter l'exécution de la fonction en renvoyant la valeur de k qui est recopiée dans a.
• L'environnement de la fonction main ne comprend que la variable a : on ne peut pas utiliser les variables i, j et k dans la fonction main.
• L'environnement de la fonction b comprend les variables i,j et k : on ne peut pas utiliser a dans la fonction b.
• La séparation des environnements permet de mieux structurer les applications.
• %i, a associé à printf (f : afficher avec le format), permet d'afficher a, sous la forme d'un entier (i). Pour faire afficher un flottant, on aurait mis %f.

Spécificateurs de format pour printf

SYMBOLE	TYPE	IMPRESSION COMME
%d ou %i	int	entier relatif
%u	int	entier naturel (unsigned)
%o	int	entier exprimé en octal
%x	int	entier exprimé en hexadécimal
%c	int	caractère
%f	double	rationnel en notation décimale
%e	double	rationnel en notation scientifique
%s	char*	chaîne de caractères

Exécution :
Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
Le resultat vaut: 25
```

Une autre fonction :

```
#include <stdio.h>
#include <stdlib.h>
```

```
int b(int i)
{
   int k;
   int s = 0;
   for(k = 1;k <= i; k++) s = s + k*k;
   return s;
}
int main()
{
   int a;
   a = b(4);
   printf("Le resultat vaut: %i\n",a);
   return 0;
}
```

Explications :
• On définit une fonction b qui a un paramètre entier i et qui calcule la somme des carrés des i premiers entiers.
• Dans la fonction main, on appelle b(4) et on récupère la valeur renvoyée dans la variable a. a contient donc la somme des 4 premiers carrés.

Exécution :
Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
Le résultat vaut: 30
```

Une fonction manipulant des réels :

```
#include <stdio.h>
#include <stdlib.h>

double b(double x, double y)
{
   double m;
   m = (x + y) / 2;
   return m;
}
int main()
{
   double a;
   a = b(3.2, 4.2);
   printf("Le resultat vaut: %f\e",a);
   return 0;
}
```

Explications :
• Dans cet exemple, la fonction b a 2 paramètres de type double, nommées x et y. La fonction b renvoie un double. Cette fonction renvoie la moyenne de x et de y. La variable m est une variable locale à la fonction b.
• Dans le programme principal, on récupère dans a la moyenne de 3.2 et 4.2 en appelant la fonction b.

Exécution :
Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
Le résultat vaut : 3.700000e+000
```

V) <u>EXEMPLES</u> :

<u>Exemple 1</u> : Calcul de la moyenne de 2 nombres.

Saisissez sous Code::Blocks le programme en C, puis testez-le. Changez certains éléments (**;** , **types**,…)
Ecrivez un programme équivalent en python, puis testez-le.

Programme en C :

```
#include <stdio.h>
#include <stdlib.h>

float moy(float a, float b)
{
   float m=(a+b)/2;
   return m;
}

void main()
{
   printf("la moyenne est de %f\n",moy(12.0,15.0));
}
```

Programme en Python :

<u>Exemple 2</u> : On souhaite créer une fonction qui aura comme attributs 2 nombres réels et 1 nombre entier.
Si le nombre entier est inférieur à 10, on retournera la somme et la différence des 2 nombres réels.
Si le nombre entier est supérieur ou égal à 10, on retournera le produit et le carré de la somme des 2 nombres réels.

Saisissez sous Code::Blocks le programme en C, puis testez-le. Changez certains éléments (**;** , **types**,…)
Ecrivez un programme équivalent en python, puis testez-le.

Programme en C :

```
#include <stdio.h>
#include <stdlib.h>

void calc(float a, float b, int ne)
{
   float s=a+b;
   float d=a-b;
   float p=a*b;
   float c=(a+b)*(a+b);

   if (ne<10)
   {
      printf("la somme est %f\n",s);
      printf("la difference est %f\n",d);
   }
   else
   {
      printf("le produit est %f\n",p);
      printf("le carre de la somme est %f\n",c);
   }
}

void main()
{
   calc(10.3,5.4,8);
   calc(10.3,5.4,58);
   float i=10.3;
   float j=5.4;
   int k=8;
   calc(i,j,k);
   k=58;
   calc(i,j,k);
}
```

Programme en Python :

Exemple 3 : On souhaite créer une fonction qui permettra d'échanger les 2 valeurs passées en paramètres.

Saisissez sous Code::Blocks le programme en C, puis testez-le. Changez certains éléments (**;** , **types**,…)
Ecrivez un programme équivalent en python, puis testez-le. **Aide** : pensez à utiliser des variables globales (instruction global).

Programme en C :

```
#include <stdio.h>
#include <stdlib.h>

void echange(int *x,int *y)
{
   int c=*x;
   *x=*y;
   *y=c;
}

void main()
{
   int x=6;
   int y=7;
   printf("%d  %d\n",x,y);
   echange(&x,&y);
   printf("%d  %d\n",x,y);
}
```

Programme en Python :

Version non fonctionnelle sans utilisation de pointeur (à tester).

```
void echange(int x,int y)
{
   int c=x;
   x=y;
   y=c;
}

void main()
{
   int x=6;
   int y=7;
   printf("%d  %d\n",x,y);
   echange(x,y);
   printf("%d  %d\n",x,y);
}
```

L'exemple figurant ci-dessus est très intéressant car il met en évidence l'utilisation des fameux pointeurs en langage C. Sans eux l'échange de valeur n'a pas lieu.
En effet si on ne travaille qu'avec des **variables locales**, la fonction main() va fournir à la fonction echange() la copie des 2 variables. Cette fonction va échanger les copies de ses arguments. Lorsque l'on revient dans la fonction principale, cette dernière travaille avec les variables elles-mêmes qui n'ont pas changé.
Pour résoudre ce problème, on va utiliser des pointeurs (*), ce qui va permettre de donner comme argument l'adresse mémoire des variables. La fonction echange() va elle-même travailler et modifier le contenu des adresses mémoires qui contiennent les variables. Lorsque l'on revient dans la fonction principale, cette dernière travaille avec contenu des adresses mémoires contenant les variables elles-mêmes qui ont été modifié par la fonction echange().
En Python, une solution pour contourner ce problème est d'utiliser des variables globales.

Pour avoir plus d'informations sur les pointeurs en C :
https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c/15417-a-lassaut-des-pointeurs

<u>Exemple 4</u> : On souhaite créer une fonction qui permettra de calculer les racines d'une équation du second degré et qui retournera :

- La valeur du discriminant.
- Les racines si elles existent dans R.

<u>Aide</u> : cherchez la signification de scanf.

Implémentez sous Code::Blocks le programme en C, puis testez-le.
Ecrivez un programme équivalent en python, puis testez-le.

Programme en C :

```
#include <stdio.h>
#include <stdlib.h>

void eq(int a,int b, int c)
{



}
void main()
{
   int  a;
   printf("Saisissez le coefficient a de l'equation:");
   scanf("%d",&a);



   eq(a,b,c);
}
```

Programme en Python :